Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 3

Implementation of Convolutional Neural Networks (CNNs) for Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

To design, implement, and train a Convolutional Neural Network (CNN) using TensorFlow/Keras for multi-class image classification on the CIFAR-10 dataset. To analyze feature extraction, evaluate model performance using standard metrics, and visualize learned feature maps and filters to understand the CNN's learning process.

2. Description

This experiment aims to provide hands-on experience in designing and implementing Convolutional Neural Networks (CNNs) for image classification tasks using the TensorFlow/Keras deep learning framework. The experiment begins with understanding the fundamental building blocks of CNNs, including convolutional layers, activation functions, pooling layers, flattening, and fully connected layers. Students investigate how convolution kernels extract meaningful visual features such as edges, textures, and patterns from images, while pooling operations reduce the spatial dimensions of feature maps and improve computational efficiency. The experiment also explores the influence of important CNN hyperparameters such as kernel size, stride, padding, number of filters, optimizer, batch size, and training epochs on model performance. Students visualize intermediate feature maps produced by convolutional layers to understand how CNNs progressively learn hierarchical image representations. Furthermore, they compare the effects of different pooling strategies and analyze the trainable parameters of convolutional layers. Using the CIFAR-10 image dataset containing ten object categories, students construct, train, and evaluate a CNN architecture for multi-class image classification. The performance of the model is assessed using evaluation metrics including accuracy, precision, recall, F1-score, confusion matrix, and classification report. Training and validation accuracy/loss curves, class distribution, sample images, and feature map visualizations are generated to interpret the learning behavior of the network. Through this experiment, students gain practical knowledge of CNN architecture design, feature extraction, model training, performance evaluation, and the application of deep learning techniques to computer vision problems.

3. Dataset

Dataset: CIFAR-10
Training Images: 50,000
Testing Images: 10,000
Classes: 10
Image Size: 32×32 pixels
Color Channels: 3 (RGB)

4. Experimental Procedure

Task 1: Dataset Loading and Exploration

- Load the CIFAR-10 image classification dataset using TensorFlow/Keras.
- Print the dataset dimensions and class labels.
- Display sample images from each class.
- Plot the class distribution of the dataset.

Task 2: Data Preprocessing
- Normalize pixel values to the range $[0, 1]$.
- Convert class labels to categorical format (one-hot encoding).
- Split the dataset into training and testing sets.
- Print the shapes of the processed datasets.

Task 3: CNN Model Construction
Construct a Convolutional Neural Network consisting of:
- Convolutional layers with ReLU activation.
- Max Pooling layers for spatial dimensionality reduction.
- Flatten layer.
- Fully Connected (Dense) layer.
- Output layer with Softmax activation for 10-class classification.

Display the network architecture using `model.summary()`.

Task 4: CNN Training
- Compile the CNN using an appropriate optimizer and categorical cross-entropy loss.
- Train the model for a specified number of epochs using mini-batch gradient descent.
- Record training and validation accuracy and loss after each epoch.

Task 5: Hyperparameter Analysis
- Experiment with different kernel sizes, number of filters, padding, and stride values.
- Compare different pooling strategies such as Max Pooling and Average Pooling.
- Analyze the effect of optimizer, batch size, and number of training epochs on model performance.

Task 6: Feature Map Visualization
- Extract feature maps from intermediate convolutional layers.
- Visualize the learned feature representations.
- Observe how low-level and high-level image features are progressively extracted.

Task 7: Model Evaluation
- Evaluate the trained CNN on the test dataset.
- Compute Accuracy, Precision, Recall, and F1-Score.
- Generate the Confusion Matrix and Classification Report.

Task 8: Performance Visualization
- Plot training and validation accuracy curves.
- Plot training and validation loss curves.
- Display correctly and incorrectly classified sample images.
- Summarize the overall CNN performance for the CIFAR-10 dataset.

5. Source Code

The complete source code, datasets, generated plots, and report files for this experiment are available in the following GitHub repository:

GitHub Repository:
`https://github.com/VIKASNI-S/Deep-Learning-Lab/tree/main/Lab-3`

6. Plots and their Inference

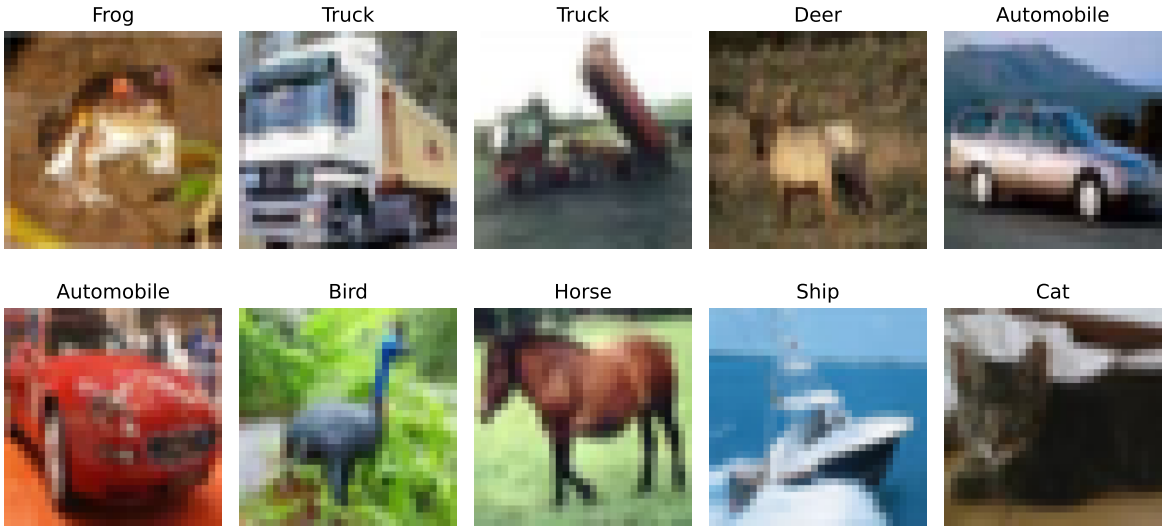

Figure 1: Sample images from the CIFAR-10 dataset representing the ten object classes.

Sample images confirm the diversity of objects present in dataset.

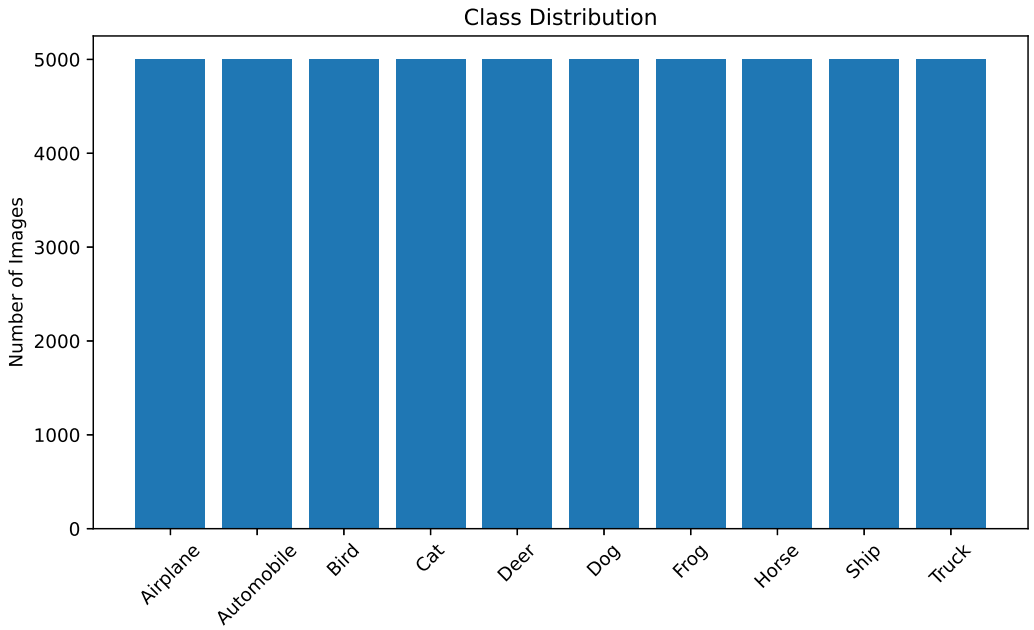

Figure 2: Distribution of images across the ten classes in the CIFAR-10 training dataset.

Dataset contains equal no.of images of all 10 classes –¿ balanced dataset. This prevents the model from being biased towads a particular class.

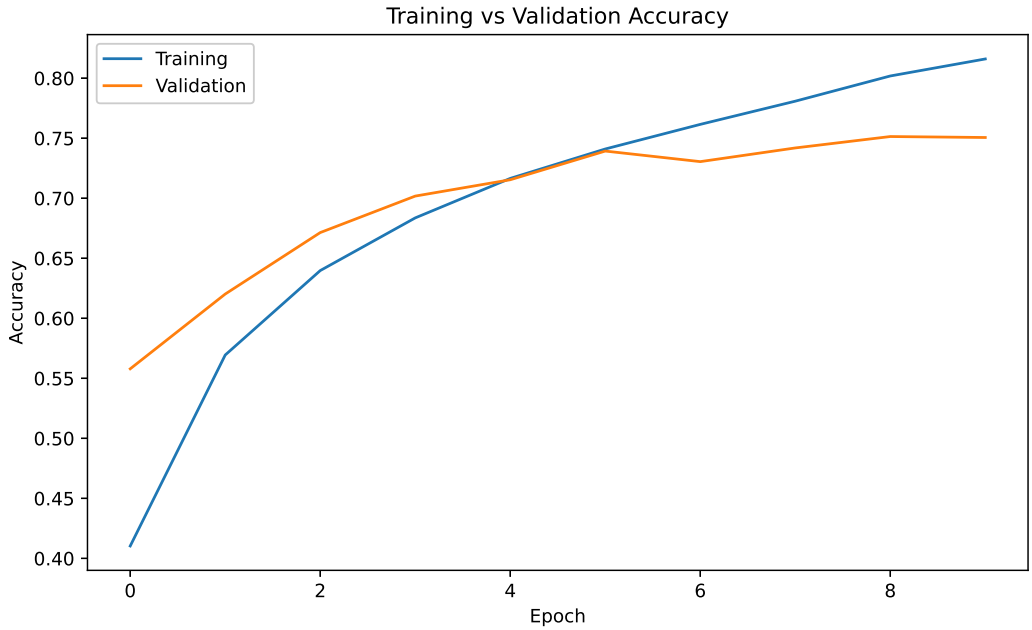

Figure 3: Training and validation accuracy over training epochs.

Training and Validation accuracy increase steadily with epochs, demonstrating that the CNN learns meaningful image features. The small gap depicts good generalization with minimal overfitting.

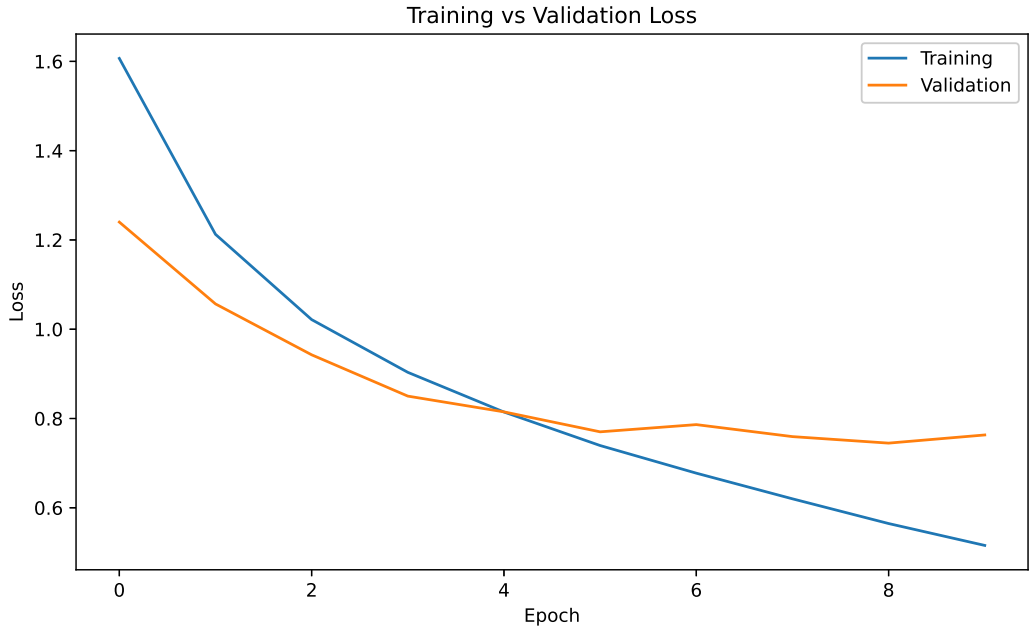

Figure 4: Training and validation loss over training epochs.

The loss decreases consistently during training, showing successful optimization of the network parameters. The validation loss follows a similar trend, indicating stable learning and good convergence.

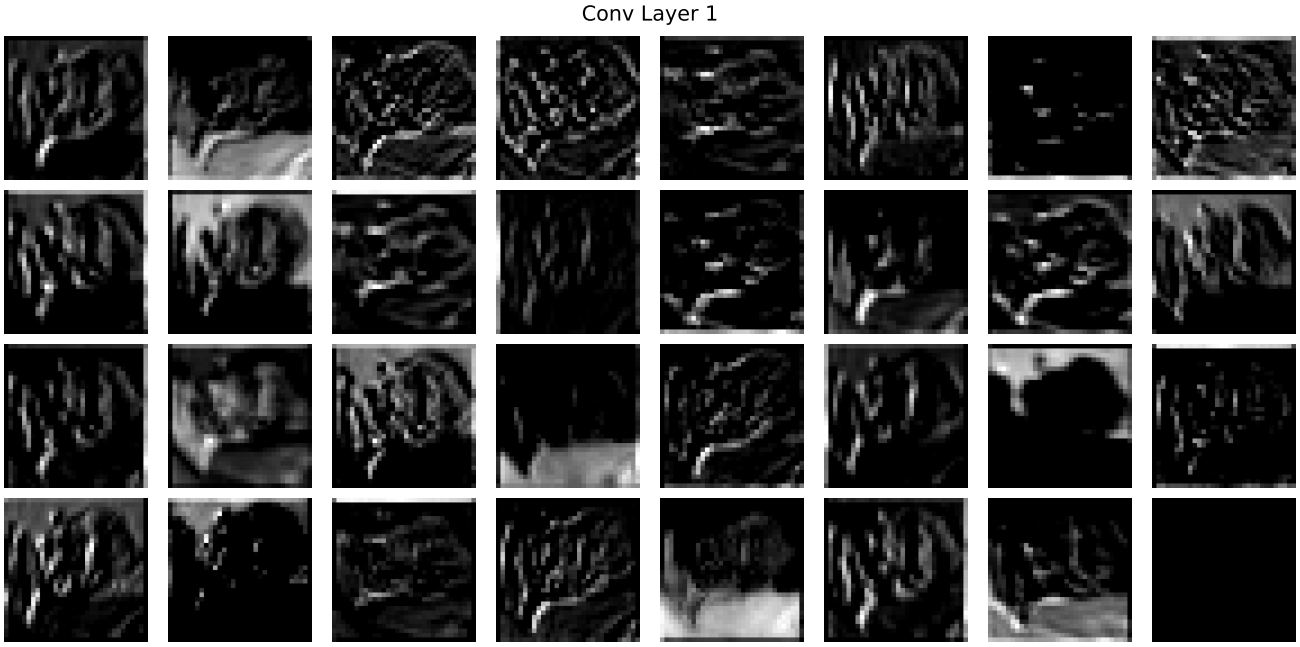

Figure 5: Feature maps generated by the first convolutional layer.

1st Convolutional layer capatues low level features such as edges and corners. They serve as basis for the deeper feature learning.

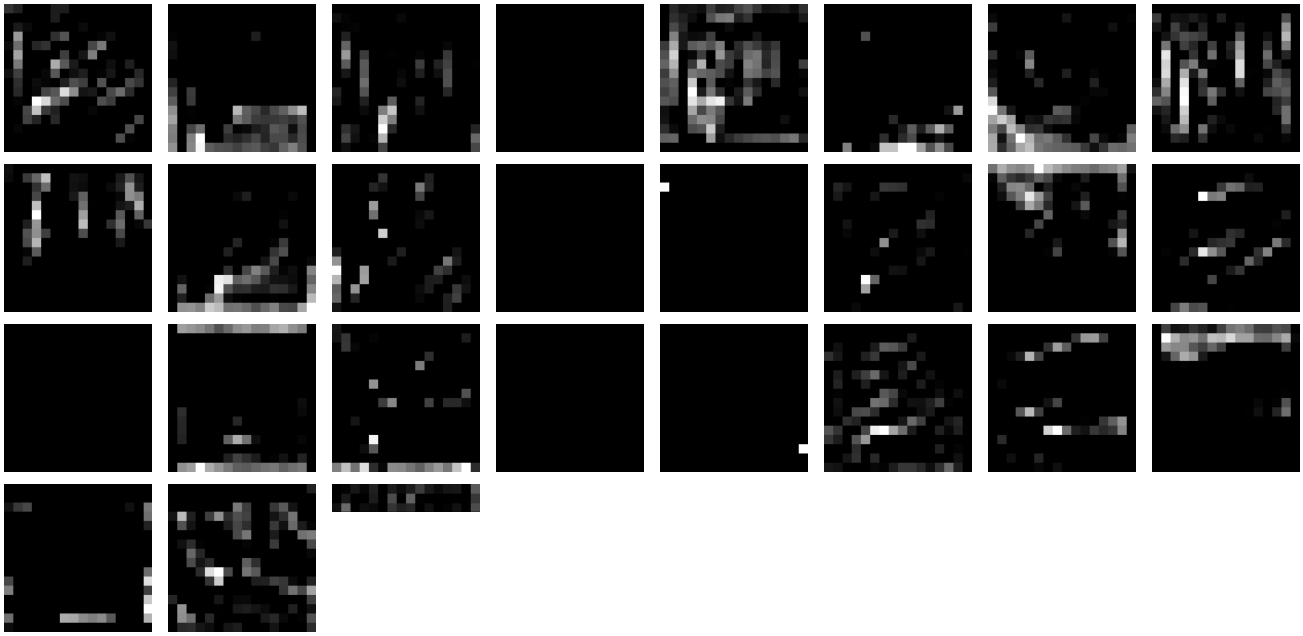

Figure 6: Feature maps generated by the second convolutional layer.

2nd Convolutional laye extracts more complex featues from the previous layer. It identifies object parts and distinctive features.

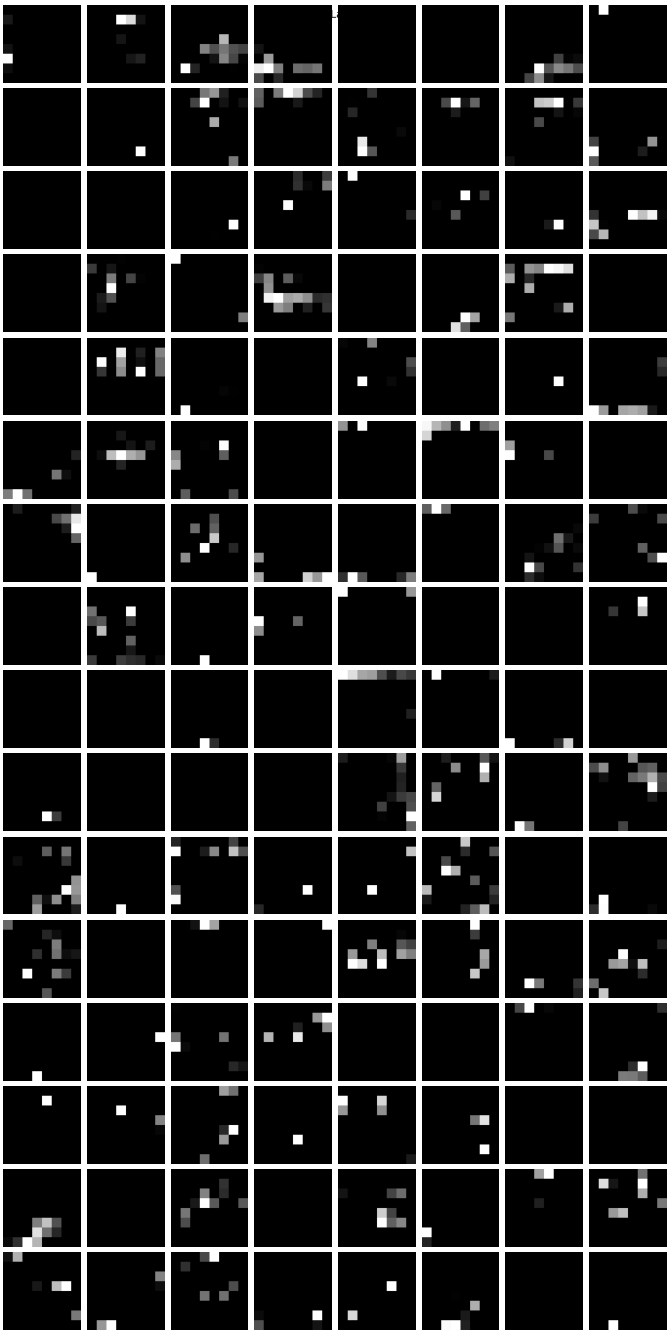

Figure 7: Feature maps generated by the third convolutional layer.

3rd Convolution layer learns high level semantic features like object shapes and class specific patterns. Also suppresses irrelevant background information.

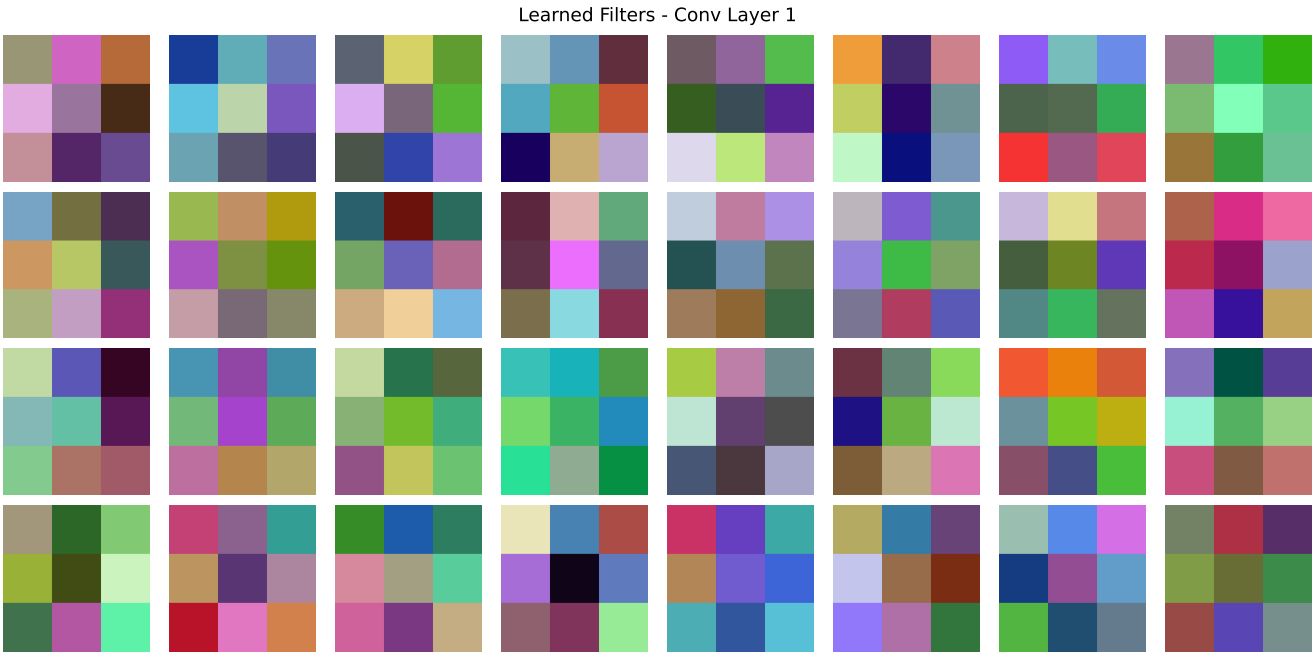

Figure 8: Learned convolution kernels of the first convolutional layer.

The learned kernels exhibit simple edge- and gradient-detecting patterns, indicating that the network has learned fundamental visual filters. These kernels form the basis for extracting low-level image features from the input.

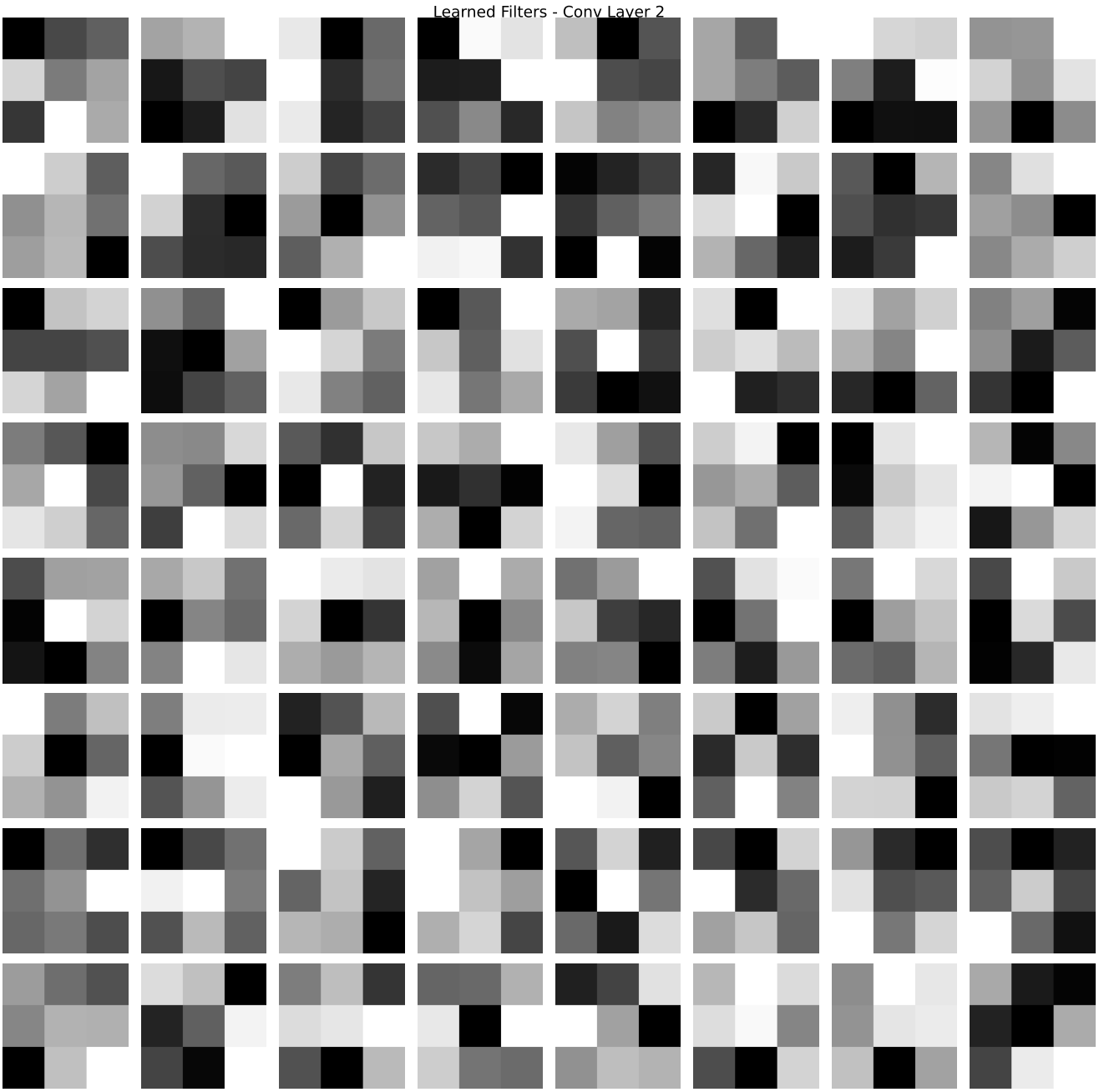

Figure 9: Learned convolution kernels of the second convolutional layer.

The kernels become more structured and complex than those in the first layer, reflecting the learning of texture- and pattern-oriented filters.

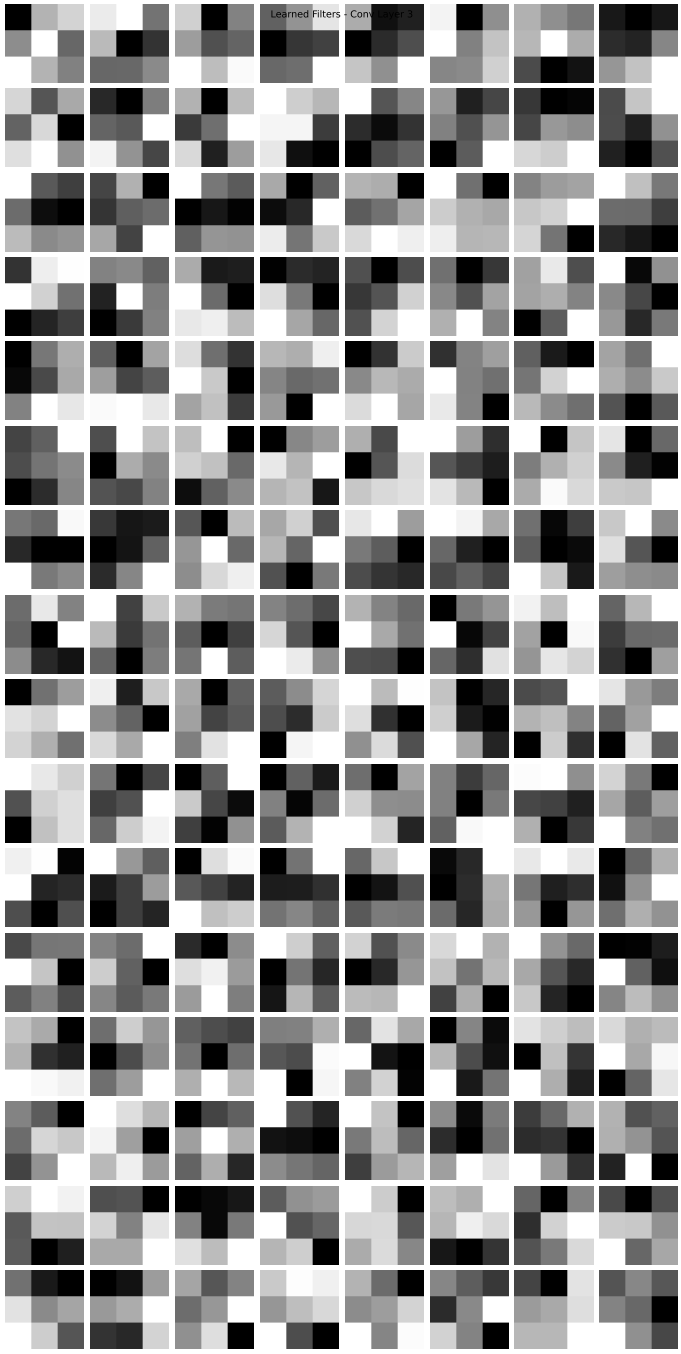

Figure 10: Learned convolution kernels of the third convolutional layer.

The learned kernels appear increasingly abstract and less visually interpretable, as they are optimized to capture high-level semantic information. These filters contribute to distinguishing different object classes during classification.

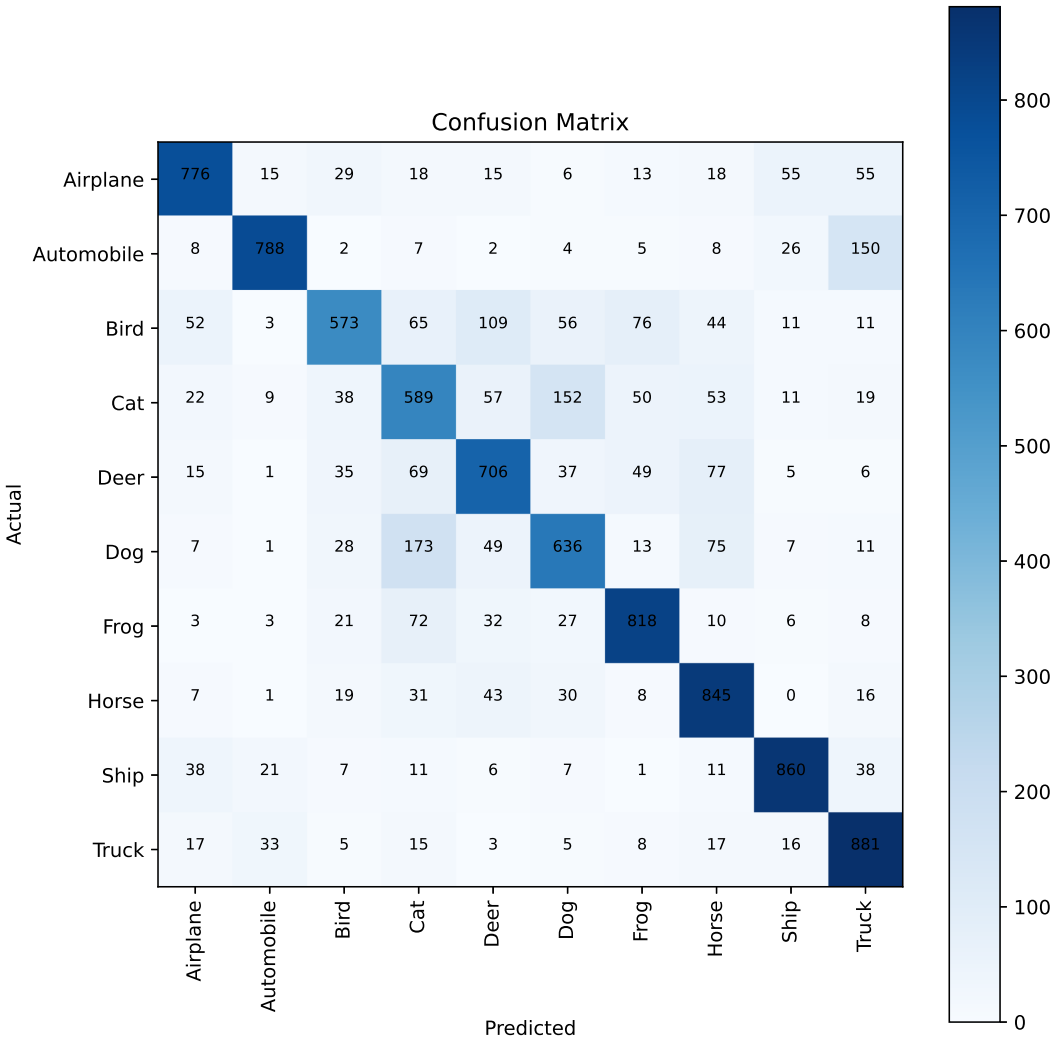

Figure 11: Confusion matrix of the CNN model on the CIFAR-10 test dataset.

Most predictions lie along the diagonal showing that the model correctly classifies the majority of images. There is chance for misclassification between visually similar classes.

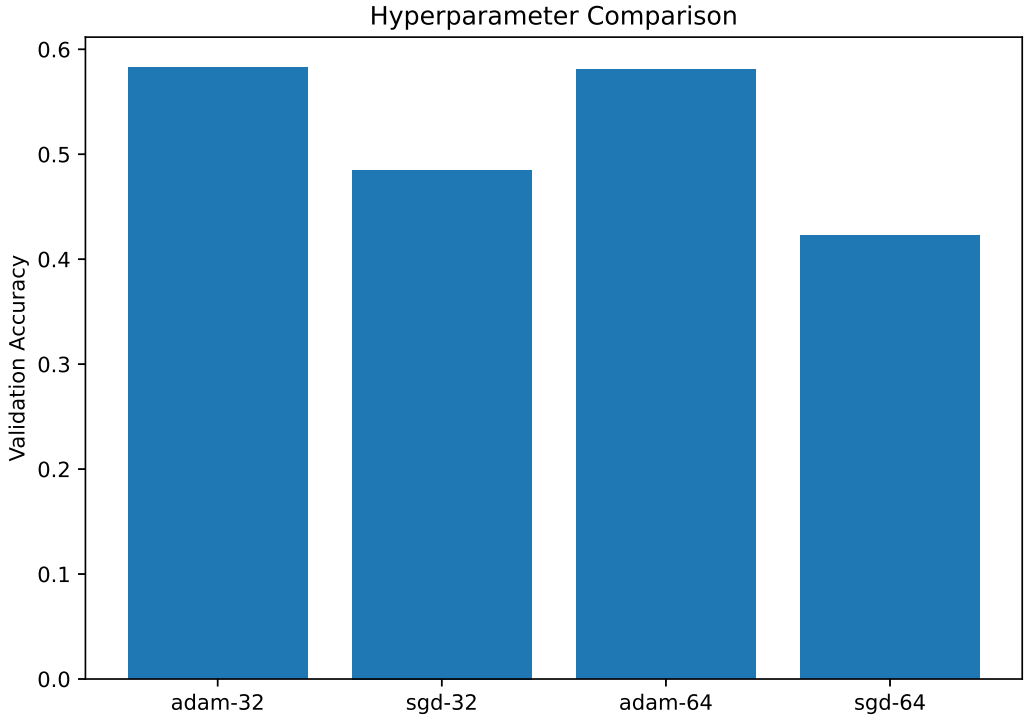

Figure 12: Validation accuracy obtained using different optimizers and batch sizes.

The model accuracy varies with different hyperparameter settings, demonstrating that hyperparameter selection significantly influences CNN performance. The optimal combination of learning rate, batch size, optimizer, and number of epochs achieved the highest classification accuracy.

7. Results

0.1 Evaluation Metrics

Table 1: Performance Evaluation Metrics of the CNN Model

Metric	**Value**
Accuracy	0.7472
Precision	0.7508
Recall	0.7472
F1-Score	0.7461

8. Classification Report

Table 2: Classification Report of the CNN Model on the CIFAR-10 Test Dataset

Class	**Precision**	**Recall**	**F1-Score**	**Support**
Airplane	0.82	0.78	0.80	1000
Automobile	0.90	0.79	0.84	1000
Bird	0.76	0.57	0.65	1000
Cat	0.56	0.59	0.57	1000
Deer	0.69	0.71	0.70	1000
Dog	0.66	0.64	0.65	1000
Frog	0.79	0.82	0.80	1000
Horse	0.73	0.84	0.78	1000
Ship	0.86	0.86	0.86	1000
Truck	0.74	0.88	0.80	1000
Accuracy	0.75			10000
Macro Average	0.75	0.75	0.75	10000
Weighted Average	0.75	0.75	0.75	10000

9. References

1. Goodfellow et al., *Deep Learning*.
2. Alex Krizhevsky, "The CIFAR-10 Dataset," Canadian Institute for Advanced Research (CIFAR), 2009.
3. TensorFlow/Keras Documentation.